Strings in Java

String, StringBuffer, and StringBuilder

Introduction

- A String is an object in Java that represents a sequence of characters.
- String is immutable (unchangeable once created).
- Strings are objects of the String class in java.lang package.

Characteristics of String

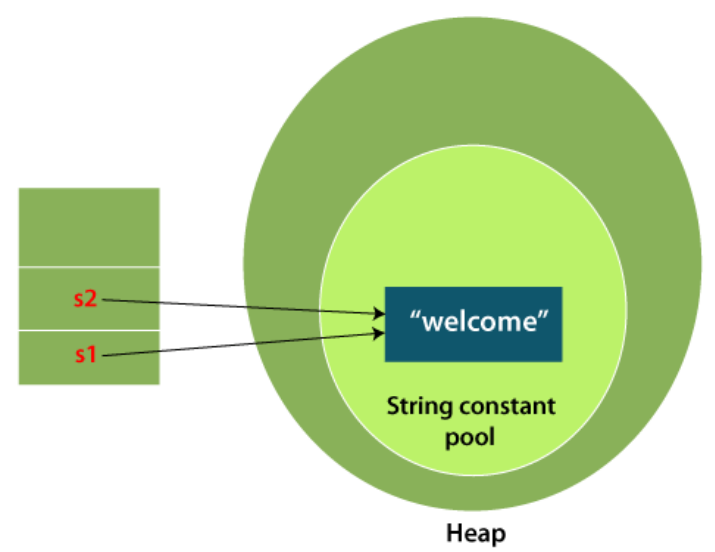
- Immutable: Once created, the string value cannot be modified.
- Stored in String Pool: Common strings are stored in the string pool for memory efficiency.
- Reference Type: String is an object, not a primitive type.
- Character Sequence: String holds Unicode characters (16-bit).

Different Ways to Declare Strings

String Literal

- This is the most common way to declare a string in Java.
- It creates a reference to the string literal “welcome", and Java stores this string in the string pool.
- If "Hello" already exists in the pool, the same reference is used.

```
String s1 = “welcome";
String s2 = “welcome";
```



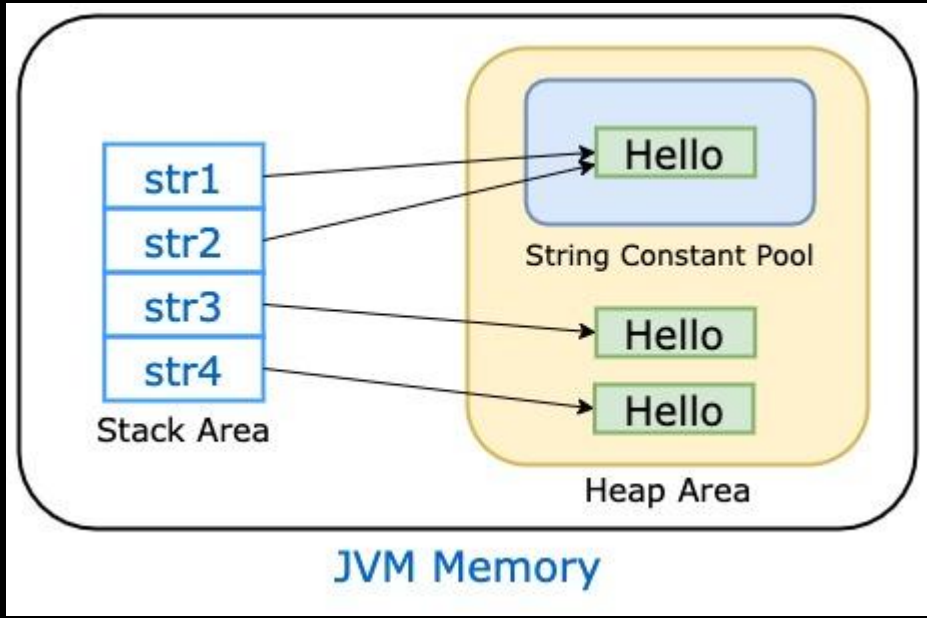
Different Ways to Declare Strings

Using new Keyword

- This creates a new String object in memory.
- So, the new keyword bypasses the string pool and creates a new string in heap memory, even if the exists in the string pool.
- This is generally less efficient.

```
String str = new String("Hello");
```

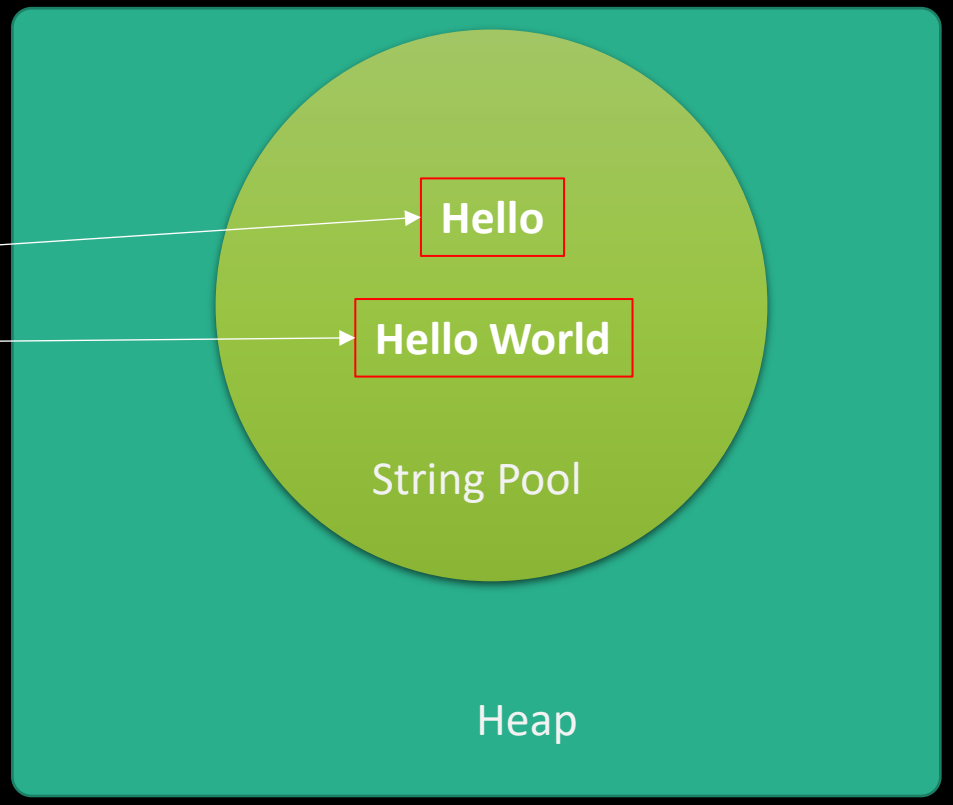
```
String str1 = “Hello”;
String str2 = “Hello”;
String str3 = new String(“Hello”);
String str4 = new String(“Hello”);
```



Immutable String

- In Java, String objects are immutable.
- Immutable simply means unmodifiable or unchangeable.

```
class Main {
  public static void main(String[] args) {
    String s1 = "Hello";
    s1 = s1+" World";
    System.out.println(s1);
}}
```



Why is String immutable?

- **Security:** Immutability ensures that string values cannot be changed, making them safe to use in situations where multiple threads might access the same string.

- **Performance:** Since strings are immutable, Java uses a string pool (interned strings) to reuse strings efficiently, avoiding the creation of multiple instances for the same value.

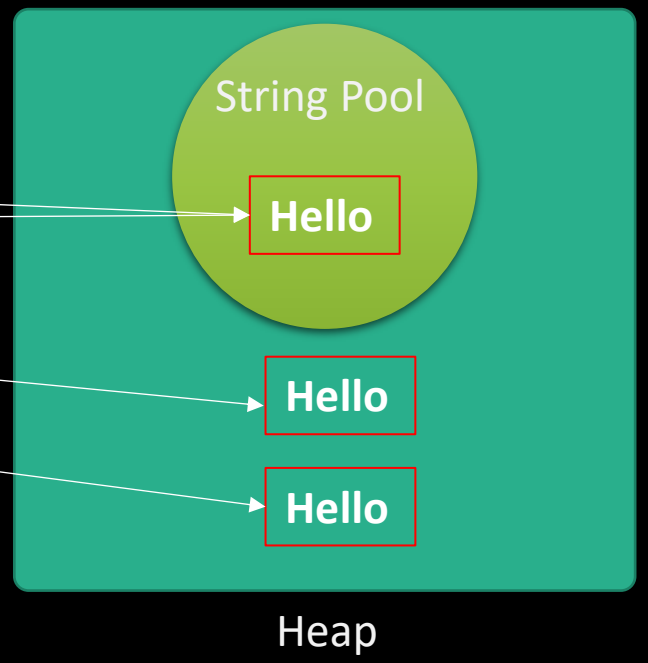
- Hashcode Caching: The immutability of strings makes it possible to cache their hashcode, improving the performance of operations like hashCode() and equals().

String Comparison (== vs equals())

- == compares references (memory location)
- equals() compares actual content (character sequence).

```
String str1 = "Hello";
String str2 = "Hello";
String str3 = new String("Hello");
String str4 = new String("Hello");

System.out.println(str1 == str2);  // true (same reference)
System.out.println(str1 == str3);  // false (different references)
System.out.println(str1.equals(str3));  // true (same content)
System.out.println(str3.equals(str4));  // true (same content)
```



Useful String Methods

- public int length() - Returns the length of the string.
- public char charAt(int index) - Returns the character at a specified index.
- public String substring(int beginIndex, int endIndex) - Extracts a substring.
- public String toUpperCase() - Converts the string to uppercase.
- public String toLowerCase() - Converts the string to lowercase.
- public String trim() - Removes leading and trailing whitespace.
- public String replace(char oldChar, char newChar) - Replaces a character or substring.
- int indexOf(String str) - Finds the first occurrence
- int lastIndexOf(String str) - Finds the last occurrence

Useful String Methods

- public boolean startsWith(String prefix) – checks whether the String starts with specified string.
- public boolean endsWith(String suffix) - checks whether the String ends with specified string.
- public boolean equals(Object anObject) - compares two strings, and returns true if the strings are equal, and false if not.
- public boolean equalsIgnoreCase(String anotherString) - Compares two strings for equality ignoring case.
- public int compareTo(String anotherString) - compares two strings lexicographically. returns 0 if equal, <0 if current string is less, and >1 if current string is greater.
- public String[] split(String regex) - Divides a string into an array of substrings based on matches of a provided regular expression (regex).

StringBuffer

- StringBuffer is used for mutable strings.
- It is thread-safe, meaning multiple threads can use it safely.
- Synchronized methods ensure thread safety but may slow down performance in single-threaded environments.

```
StringBuffer str1 = new StringBuffer("Hello");
str1.append(" World");  // Adds " World" to the string
System.out.println(str1);  // Output: "Hello World"
```

Important Methods of StringBuffer

- StringBuffer append(String str) - Adds data to the end of the current string.
- StringBuffer insert(int offset, String str) - Inserts data at a specified position.
- StringBuffer delete(int start, int end) - Removes characters from start index to end index (end index exclusive).
- StringBuffer reverse() - Reverses the content of the string.
- StringBuffer replace(int start, int end, String str) - substitute a specific range of characters within the buffer with a new string. (end exclusive)

StringBuilder

- StringBuilder is also used for mutable strings.
- Not thread-safe, but offers better performance in single-threaded environments.

```
StringBuilder builder = new StringBuilder("Hello");
builder.append(" World");
System.out.println(builder);  // Output: "Hello World"
```

Note: In Java, **thread-safe** describes code, classes, or methods that behave correctly when accessed by multiple threads simultaneously.

Important Methods of StringBuilder

The StringBuilder class in Java provides several methods to modify its contents in-place, making it more efficient than the immutable String class for frequent changes.

- append(data) - Adds text to the end of the string.
- insert(int offset, data) - Inserts text at a specific position.
- delete(int start, int end) - Removes text from the string.
- reverse() - Reverses the content of the string.
- replace(int start, int end, String str) - Replaces characters from start to end-1 with the given string.

StringBuffer vs StringBuilder

Feature	StringBuffer	StringBuilder
Thread Safety	Thread-safe (synchronized)	Not thread-safe
Performance	Slower (due to synchronization overhead)	Faster (no synchronization)
Use Case	Used in multi-threaded environments	Used in single-threaded or performance-critical applications